

Cyber Kill Chain

Developed by **Lockheed Martin** in 2011, the Cyber Kill Chain is a model that breaks down a cyberattack into sequential stages — borrowed from military terminology (the idea of "breaking the kill chain" to stop an attack before it completes). It's one of the earliest and most influential frameworks for understanding *how* attackers operate, stage by stage.

The core idea: **an attacker must succeed at every stage to achieve their objective. A defender only needs to break ONE link to stop the entire attack.**

The 7 Stages

1. Reconnaissance

The attacker gathers information about the target before ever touching their systems.

What happens:

- OSINT gathering — LinkedIn employee names, job postings (reveals tech stack), company website
- WHOIS lookups, DNS enumeration
- Social media scraping
- Identifying exposed services via Shodan/Censys
- Email harvesting for phishing targets

Why it matters: This is passive and often invisible to the target. Most real-world breaches start here, sometimes weeks or months before the actual intrusion.

Analogy: A burglar walking around a house, noting which windows are unlocked, when the owners leave for work, and where the dog sleeps.

2. Weaponization

The attacker couples an exploit with a payload to create a deliverable weapon — this happens entirely on the attacker's side, not yet touching the victim.

What happens:

- Pairing a malicious macro with a Word doc

- Embedding a Remote Access Trojan (RAT) into a PDF exploit
- Creating a malicious payload with tools like Metasploit's `msfvenom`
- Building a phishing kit that mimics a login page

Why it matters: This stage is invisible to defenders since it happens in the attacker's lab — no telemetry exists yet.

3. Delivery

The weapon is transmitted to the target environment.

Common delivery vectors:

- Phishing emails (most common — ~90%+ of breaches start here)
- Malicious USB drives (dropped in parking lots — "USB drop attack")
- Watering hole attacks (compromising a website the target frequently visits)
- Drive-by downloads
- Direct exploitation of an internet-facing service

Defensive opportunity: Email gateways, spam filters, web proxies, and user awareness training all operate here. This is one of the **best points to stop an attack cheaply**, since it hasn't executed yet.

4. Exploitation

The weapon's code triggers, exploiting a vulnerability to gain code execution on the victim system.

What happens:

- A macro executes and downloads a second-stage payload
- A buffer overflow in a vulnerable service gets triggered
- A user clicks a link and a browser exploit fires
- An exposed RDP/SMB service gets exploited (e.g., EternalBlue for MS17-010)

This is the moment the attacker moves from "outside" to "inside."

5. Installation

The attacker installs persistence mechanisms so they don't lose access if the system reboots or the user closes a file.

What happens:

- Installing a backdoor or RAT
- Creating scheduled tasks or registry run keys (Windows persistence)
- Adding a malicious service
- Creating new user accounts
- Installing web shells on compromised servers

Defensive opportunity: EDR (Endpoint Detection & Response) tools are designed specifically to catch this stage — unusual process creation, registry changes, new services.

6. Command & Control (C2)

The compromised system "phones home" to the attacker's infrastructure, establishing a channel for remote control.

What happens:

- Beaconsing to a C2 server (Cobalt Strike, Metasploit, custom malware)
- Often disguised as legitimate traffic (HTTPS, DNS tunneling, using Twitter/Telegram as C2 channels to blend in)
- Attacker now has interactive remote access

Defensive opportunity: This is where **SOC analysts** (your target role) shine — detecting beaconsing patterns, unusual outbound connections, DNS anomalies, and traffic to known-bad IPs/domains via SIEM correlation.

7. Actions on Objectives

The attacker achieves their actual goal — this is why they broke in.

Depends on attacker motive:

- **Data exfiltration** — stealing intellectual property, customer data, credentials
- **Lateral movement** — pivoting deeper into the network (often overlaps with this stage — using tools like Mimikatz, pass-the-hash, or exploiting AD)

misconfigurations)

- **Ransomware deployment** — encrypting files for extortion
- **Destruction** — wiping data (e.g., NotPetya)
- **Espionage** — long-term persistent access for intelligence gathering

Visual Summary

```
Recon → Weaponization → Delivery → Exploitation → Installation → C2 → Actions on Objectives
|           |           |           |           |           |           |
| OSINT      Payload     Phishing    Code exec   Backdoor    Beacon     Exfil/Ransomwa
| re         creation    email       |           |           | | |
| build      |           |           |           |           |
| nt         |           |           |           |           |
|           |           |           |           |           |
```

Why This Model Matters for Study

1. It's a defense-in-depth teaching tool

Every stage represents a chance to detect or block the attack. This is the foundation of the "Defender only needs to win once" philosophy — as opposed to the attacker, who needs every stage to succeed.

2. It maps directly to blue-team tooling

Kill Chain Stage	Defensive Control
Recon	Threat intel, attack surface monitoring
Weaponization	(mostly invisible — sandbox/malware analysis after the fact)
Delivery	Email gateway, spam filter, web proxy
Exploitation	Patch management, IPS, WAF
Installation	EDR/AV, application whitelisting
C2	SIEM, network monitoring, DNS filtering
Actions on Objectives	DLP, network segmentation, honeypots

3. It connects to MITRE ATT&CK

The Kill Chain is high-level and linear. **MITRE ATT&CK** is the more granular, non-linear evolution — it breaks each of these stages into dozens of specific

Tactics, Techniques, and Procedures (TTPs). Think of the Kill Chain as the "chapters" and ATT&CK as the detailed "paragraphs" within each chapter. Since you're studying SOC/pentesting, ATT&CK is what you'll use day-to-day for actual technique mapping — the Kill Chain gives you the conceptual skeleton.

Known Limitations (worth understanding, not just memorizing)

- **It's linear**, but real attacks are often non-linear — attackers loop back (e.g., re-doing recon after initial access to find more targets internally)
 - **It was designed for perimeter-based, malware-driven attacks** — it doesn't model well against:
 - Insider threats (no "delivery" needed)
 - Web application attacks (no clear "installation" stage in something like SQLi)
 - Living-off-the-land attacks (using legitimate tools like PowerShell — blurs "exploitation" and "installation")
 - **This is why the Unified Kill Chain (2017) and MITRE ATT&CK emerged** — to address these gaps with more flexible, non-linear models
-